AusFlux

A Cloud-Native Pipeline for Cross-Platform Sentiment Analysis of Australian Cities

COMP90024 Cluster and Cloud Computing — Assignment 2

The University of Melbourne, Semester 1, 2026

Team 47

Xianglong Fu	xianglong.fu@student.unimelb.edu.au
Muchuan Ji	muchuan.ji@student.unimelb.edu.au
Zhuo Wen	zwen5515@student.unimelb.edu.au
Zeming Ji	zeming.ji@student.unimelb.edu.au
Chang Gu	changgu@student.unimelb.edu.au

May 2026

Abstract

This report presents a cloud-native data analytics application built on the Melbourne Research Cloud (MRC) for the COMP90024 Cluster and Cloud Computing assignment. The application ingests, indexes, and exposes for analysis a continuously growing corpus of social-media posts from two distinct platforms (Mastodon and BlueSky) alongside daily weather observations for four Australian cities (Melbourne, Sydney, Brisbane, Perth) from the Open-Meteo API.

The system is composed of four cooperating service layers deployed on a four-node Kubernetes cluster: a serverless ingestion layer implemented as Fission timer-triggered functions; an Elasticsearch storage layer using three explicitly mapped indices; a FastAPI-based RESTful service layer exposing fourteen endpoints; and an analytical client implemented as a Jupyter notebook. Two engineering contributions are highlighted: a *dual-mode data access pattern* in which the notebook switches transparently between direct Elasticsearch access and HTTP access through the REST API via a single import flag; and a *persistent JWT cache* backed by a Kubernetes ConfigMap with RBAC-scoped access, which resolves BlueSky's per-account session rate-limiting and increased harvested volume approximately six-fold.

At the time of writing the system has indexed 15,975 Mastodon posts, 17,560 BlueSky posts, and 1,496 daily weather records, with the streaming pods having run continuously for 21+ days without restart. Two research questions on weather–sentiment correlation and cross-platform discourse comparison are addressed as illustrative analyses that exercise the deployed system end-to-end.

Keywords: Kubernetes; Fission; Elasticsearch; Serverless; Sentiment Analysis; VADER; Mastodon; BlueSky; ELT; RBAC; Cloud-Native Architecture.

Contents

1 Introduction **4**

1.1 Application Context . . . 4
1.2 System Scope . . . 4
1.3 Research Questions . . . 4
1.4 Engineering Highlights . . . 5
1.5 Report Structure . . . 5

2 System Architecture **5**
2.1 Architectural Overview . . . 5
2.2 Dual-Mode Data Access . . . 6

3 Cluster Deployment **7**
3.1 Provisioning on Melbourne Research Cloud . . . 7
3.2 Declarative Resource Management . . . 7
3.3 Storage Layer: ECK and Persistent Volumes . . . 8
3.4 Day-to-Day Operational Commands . . . 8

4 Data Delivery **9**
4.1 Data Sources . . . 9
4.2 Elasticsearch Schema . . . 9
4.3 Two-Phase Data Harvesting Strategy . . . 10
4.4 Data Volume and Coverage . . . 11

5 Implementation Details **12**
5.1 RESTful API Design . . . 12
5.2 Dual-Mode Data Access: Implementation . . . 13
5.3 Shared Utility Modules . . . 13

6 Automated Testing and Continuous Deployment **13**
6.1 Test Suite Organisation . . . 14
6.2 The FakeES Mock Pattern . . . 14
6.3 GitLab CI/CD Pipeline . . . 15
6.4 What the Pipeline Verifies (and What It Does Not) . . . 15

7 Frontend: Analytical Scenarios and Results **15**
7.1 Scenario Selection Rationale . . . 15
7.2 Scenario 1: Weather–Sentiment Correlation . . . 16
7.2.1 Methodology . . . 16
7.2.2 Findings . . . 16
7.3 Scenario 2: Cross-Platform Discourse Comparison . . . 17
7.3.1 Methodology . . . 17
7.3.2 Findings . . . 19
7.4 Synthesis Across Scenarios . . . 22

8 Discussion **22**
8.1 Error Handling and Engineering Challenges 22
8.2 Fault Tolerance and Single Points of Failure 24
8.3 Pros and Cons of the Cloud Technology Stack 24
8.4 Limitations and Future Work 25

9 Conclusions **25**

10 Team Contribution **26**

11 LLM Usage Declaration **26**

A Source Code Repository **28**

B YouTube Video Link for Demonstration **28**

C Elasticsearch Mappings **28**
C.1 posts_mastodon 28
C.2 posts_bluesky 29
C.3 weather 29

D Sample Elasticsearch Queries **30**
D.1 Selection-Bias Filter for Cross-Platform Comparison 30
D.2 Sentiment Distribution Aggregation 30
D.3 City-Date Sentiment Aggregation (RQ1 Preparation) 31
D.4 Deep Pagination via Point-in-Time 32

E Repository Structure **32**

1 Introduction

1.1 Application Context

Modern cloud-native applications increasingly rely on the composition of stateless services running in container-orchestration platforms such as Kubernetes, with data persistence delegated to specialised indexing engines and analytical workloads communicating with the storage layer through well-designed RESTful interfaces. In this project we build such a system end-to-end on the Melbourne Research Cloud (MRC) [2], exercising every layer from cluster onboarding through serverless function development, indexed storage design, RESTful API specification, and downstream analytical consumption.

Our team chose as the application domain the intersection of **social-media sentiment and meteorological conditions in Australian cities**. This domain satisfies three constraints the assignment imposes on the data: (i) it requires multiple independent streaming data sources, motivating a multi-function ingestion design; (ii) it exhibits a natural cross-platform comparison angle, justifying separate storage schemata and platform-aware API endpoints; and (iii) the data evolves continuously, demonstrating that the streaming layer functions as intended over the assessment window rather than only at deployment time.

1.2 System Scope

The deployed application performs the following functions:

- **Continuous data ingestion.** Three Fission serverless functions, scheduled by Fission timers, harvest posts from Mastodon and BlueSky, and daily weather observations from Open-Meteo. A one-shot historical-backfill pipeline pre-populated the indices on day one, after which the streaming functions maintained near-real-time growth.
- **Indexed storage.** A three-index Elasticsearch design (`posts_mastodon`, `posts_bluesky`, `weather`) provides full-text search and aggregation capabilities. Sentiment scoring (VADER) is applied at ingestion time and persisted with each post, so that aggregations require no re-computation.
- **RESTful service exposure.** A FastAPI application deployed as a Kubernetes Deployment exposes the indexed data through fourteen endpoints covering raw retrieval, aggregations, time-series, cross-platform comparisons, and a self-describing OpenAPI specification at `/docs`.
- **Analytical consumption.** A Jupyter notebook serves as the analytical client, consuming the indexed data through either of two interchangeable access modules and producing the figures presented in Section 7.

1.3 Research Questions

RQ1. To what extent do weather conditions correlate with daily aggregate social-media sentiment in each of the four target Australian cities?

RQ2. How do users on Mastodon and BlueSky differ in their sentiment expression and discourse vocabulary within the Australian social-media context?

A third research question on user demographics was descoped during planning, as neither source API exposes the necessary attributes. Findings for RQ1 and RQ2 are presented in Section 7 as concrete evidence that the deployed system supports realistic analytical workloads.

1.4 Engineering Highlights

Two design choices distinguish this implementation from a baseline solution to the assignment brief:

1. **A dual-mode data-access architecture** (Section 2.2) consisting of two Python modules with identical public interfaces: one performing direct Elasticsearch access using Point-in-Time pagination, the other performing HTTP access through the REST API. The analytical notebook switches between them through a single import line, making the RESTful interface a live and demonstrable path rather than a token gesture.

2. **A persistent JWT cache for rate-limited APIs** (Section 8.1) implemented as a Kubernetes ConfigMap with an RBAC-scoped service-account. This resolves BlueSky's per-account `createSession` rate-limiting using only native Kubernetes primitives, without introducing an additional database service to the architecture.

1.5 Report Structure

Section 2 describes the overall system architecture, the deployment topology, and the cooperation patterns between service layers. Section 3 details the cluster provisioning, declarative resource management, and the dynamic-scaling demonstration. Section 4 describes the data sources, the two-phase harvest strategy, and the streaming pipeline. Section 5 covers the Elasticsearch schema, the RESTful API surface, and the dual data-access modules. Section 8.1 consolidates the engineering challenges encountered, the fault-tolerance design, and the single points of failure. Section 6 describes the automated test suite and continuous-integration pipeline. Section 7 presents the two analytical scenarios and their findings. Section 8 reflects on the strengths and friction points of the cloud technology stack. Section 8.4 addresses limitations and future work. Sections 9 and 11 conclude and declare the team's use of large-language-model assistance.

2 System Architecture

2.1 Architectural Overview

The system is a layered cloud-native data pipeline deployed on the Melbourne Research Cloud (MRC). Four OpenStack VMs form a Kubernetes v1.34.6 cluster: one control-plane node and three workers. Application workloads are organised into three namespaces, each with a single responsibility: `fission` hosts the ingestion functions (Section 4); `elastic` hosts the Elasticsearch storage layer (Section 4.2); and `default` hosts the FastAPI REST service (Section 5.1). Cross-layer communication uses cluster-internal DNS service discovery and never leaves the cluster boundary.

Persistent state is confined to two locations: the Elasticsearch indices, backed by a Cinder PersistentVolume; and the cluster's own `etcd`, managed by the control plane. The Fission functions and

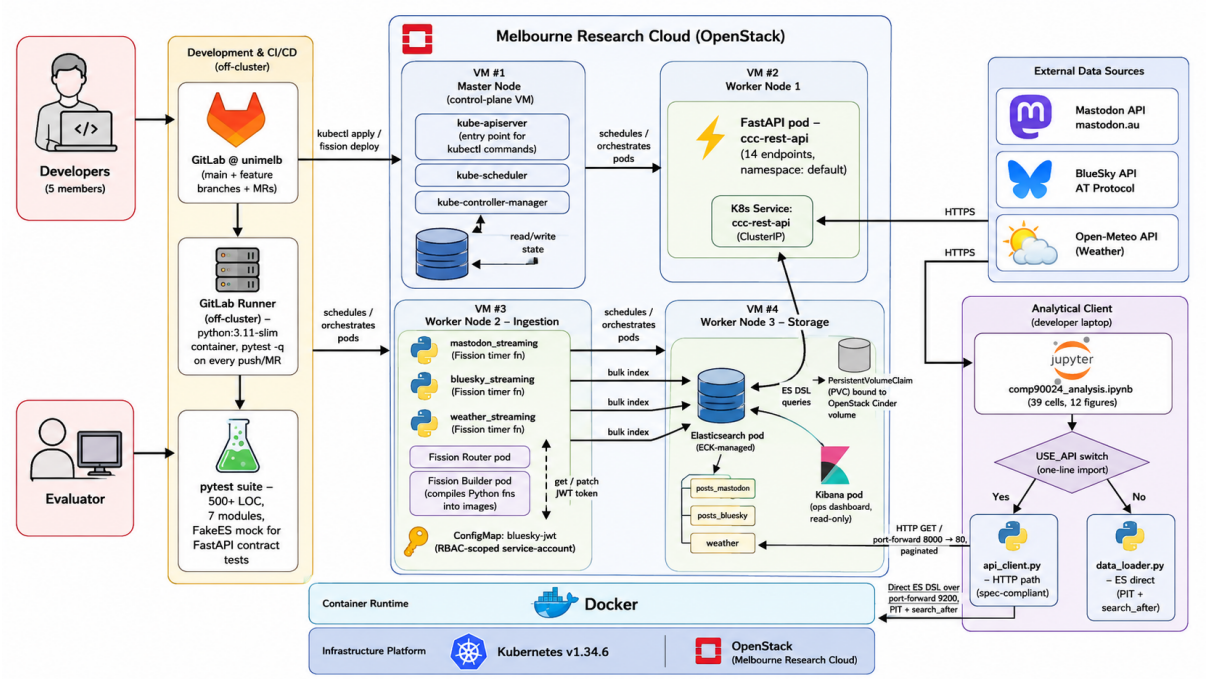


Figure 1: End-to-end system architecture deployed on the Melbourne Research Cloud (OpenStack). A four-node Kubernetes cluster (one control-plane VM and three worker VMs) hosts the application: Worker Node 1 runs the FastAPI service exposing fourteen REST endpoints; Worker Node 2 runs the Fission control plane and the three timer-triggered ingestion functions alongside the ConfigMap-backed BlueSky JWT cache; Worker Node 3 runs the ECK-managed Elasticsearch cluster with three indices on a PersistentVolume backed by OpenStack Cinder. Off-cluster, GitLab triggers a CI pipeline on every push that executes the pytest suite inside a Python 3.11 container. The analytical Jupyter notebook accesses the data through one of two interchangeable Python modules (`api_client.py` via HTTP through the API, or `data_loader.py` via direct PIT-paginated Elasticsearch), selected through a single import-time flag. All external access to cluster services is mediated through authenticated `kubectl port-forward` sessions.

the FastAPI service are stateless and freely replaceable. One small exception, the BlueSky session-token state, is held in a Kubernetes ConfigMap rather than in a separate database; the design of that ConfigMap-backed cache is described in Section 8.1. The deployment supports runtime extension: additional harvester functions can be added without modifying the storage or serving layers.

2.2 Dual-Mode Data Access

The most distinctive architectural feature of this system is that the analytical client (the Jupyter notebook) can reach the data through two interchangeable paths: the REST API, or a direct Elasticsearch connection using Point-in-Time pagination. Both paths are implemented as Python modules with identical public interfaces, and the notebook switches between them through a single import flag. The two paths apply the same filters and produce identical analytical output; only execution time differs.

This dual-mode design makes the RESTful interface a live, demonstrable component rather than a token gesture: the same notebook, switched to HTTP mode, reproduces every figure in Section 7 and confirms that each API endpoint relied upon by the analyses functions correctly. The implementation of the two modules is described in Section 5.2.

3 Cluster Deployment

This section describes how the Kubernetes cluster underlying the application was provisioned, configured, and operated. The work described here was carried out by the team's designated cluster owner.

3.1 Provisioning on Melbourne Research Cloud

The cloud environment was provisioned on the Melbourne Research Cloud (MRC). The required local tools included OpenStack command-line clients, `jq`, `kubectl`, and Helm. After logging into the MRC Dashboard, the team selected the project, downloaded the OpenStack RC file, sourced it in the local terminal, and used the OpenStack CLI to provision a Kubernetes cluster [1] for the assignment.

The Kubernetes cluster was created with one control-plane node and three worker nodes, following the assignment setup pattern. After the cluster reached a healthy state, access to the cluster was configured using a kubeconfig file generated by the OpenStack Magnum command. This kubeconfig was placed under `~/.kube/config`, allowing `kubectl` to inspect cluster nodes, pods, services, logs, and running workloads. The same kubeconfig was also distributed to other team members so that they could access the shared cluster from their own development machines.

On top of the Kubernetes cluster, the main supporting services were installed. Elasticsearch was deployed using the Elastic Cloud on Kubernetes operator, while Kibana was installed to provide browser-based inspection of indices and documents. Fission was installed as the serverless framework used by the timer-triggered ingestion functions. These services formed the shared backend environment for harvesting, storing, and inspecting the social-media and weather datasets.

During development and debugging, external access to cluster services was performed through `kubectl port-forward` rather than public load balancers. For example, Elasticsearch, Kibana, and the REST API were accessed from local machines through port-forwarded localhost ports. This allowed the team to test the system securely without exposing internal cluster services directly to the public internet.

The FastAPI REST backend was then added on top of this shared environment. The API was first tested locally with Uvicorn while Elasticsearch was accessed through port-forwarding. After local validation, the API was containerised using Docker, pushed to the GitLab Container Registry, and deployed to Kubernetes as a Deployment and ClusterIP Service. In the deployed configuration, the API connects to Elasticsearch using the cluster-internal service address, while the Jupyter Notebook frontend accesses the API through:

```
kubectl port-forward svc/ccc-rest-api 8000:80
```

This setup allows the notebook to use the stable endpoint `http://localhost:8000/api` while the actual backend runs inside the Kubernetes cluster.

3.2 Declarative Resource Management

All cluster resources are managed declaratively through version-controlled YAML manifests committed to the project's `k8s/` directory in GitLab. This includes ConfigMaps, RBAC role and role-binding definitions, the API Deployment and Service, secret templates, and operator custom resources. The directory structure is summarised in Table 1.

File	Resource
`configmaps.yaml`	`bluesky-jwt` ConfigMap (session-token cache)
`rbac/bluesky-jwt-rbac.yaml`	Role and RoleBinding for Fission service account
`api-deployment.yaml`	FastAPI Deployment + Service (`ccc-rest-api`)
`secrets.example.yaml`	Template for cluster secrets (real values git-ignored)
`README.md`	Step-by-step deployment instructions

Table 1: Contents of the version-controlled `k8s/` directory.

Bringing up the application's cluster-side components after a fresh `kubectl` configuration is a sequence of `kubectl apply` commands:

3.3 Storage Layer: ECK and Persistent Volumes

Day-to-day inspection of the indexed data uses an authenticated `kubectl port-forward` session into the Elasticsearch HTTP service:

```
kubectl port-forward service/elasticsearch-es-http -n elastic 9200:9200
# In another terminal:
curl -k -u elastic:$PASSWORD https://localhost:9200/_cat/indices?v
```

The Kibana dashboard, available through `kubectl port-forward` to port 5601, provides a browser-based interface for ad-hoc inspection of indexed documents during development.

3.4 Day-to-Day Operational Commands

A small repertoire of `kubectl` and `fission` commands forms the day-to-day operational interface to the cluster:

```
# Cluster health
kubectl get nodes
kubectl get pods --all-namespaces
kubectl top nodes

# Inspect a specific pod's logs
kubectl logs -n fission <pod-name> --tail=50

# Re-deploy the API after a code change
kubectl rollout restart deployment/ccc-rest-api

# Access the REST API from a local machine
kubectl port-forward svc/ccc-rest-api 8000:80

# Test the deployed API
curl http://localhost:8000/api/health
curl http://localhost:8000/api/summary

# Inspect the BlueSky JWT ConfigMap state
kubectl get configmap bluesky-jwt -n default -o yaml
```

```
# Tail Fission function logs
fission function logs --name mastodon-streaming --follow
```

The cluster's operational stability is itself an artefact of this disciplined declarative approach. From initial provisioning through the end of the assessment window, no node has restarted unexpectedly, no Deployment has been edited imperatively, and every change to the cluster's running state is traceable to a commit in the project repository.

4 Data Delivery

4.1 Data Sources

Three external APIs supply the data ingested by the system.

Mastodon [5] is a federated micro-blogging platform on which users publish short text posts ("toots") to public timelines. We harvest from the `mastodon.au` instance through its implementation of the standard Mastodon REST API, using an OAuth Bearer token issued to a registered application account. Two endpoints are used: `/api/v1/timelines/public` for the federated public stream, and `/api/v1/accounts/{id}/statuses` for archive backfill from selected high-volume accounts.

BlueSky [3] is a decentralised social-media built on the AT Protocol. Posts are accessed through XRPC endpoints exposed by the BlueSky AppView service, principally `app.bsky.feed.searchPosts`. Authentication uses a short-lived access JWT obtained via `com.atproto.server.createSession`, refreshed periodically through `com.atproto.server.refreshSession`. The `createSession` endpoint is rate-limited per account; the caching strategy that addresses this constraint is described in Section 8.1.

Open-Meteo [6] is a free public weather API requiring no authentication. We use its historical-archive endpoint for one-shot backfill and its forecast endpoint for the rolling daily refresh, querying independently for each of the four target cities (Melbourne, Sydney, Brisbane, Perth). The daily fields harvested (minimum, mean, maximum temperature; precipitation; humidity; wind; weather condition codes) populate the `weather` index described in Section 4.2.

4.2 Elasticsearch Schema

A unified single-index design with a `platform` discriminator was considered and rejected, because Mastodon and BlueSky differ in their identity and engagement fields and forcing them into one mapping would have required either many optional fields or raw-payload storage. Instead, the deployed design uses three separate indices, each with an explicit mapping committed to the repository under `database/mappings/`: `posts_mastodon` and `posts_bluesky` for the two social-media platforms, and `weather` for the per-(city, date) weather observations.

Platform-specific identity and engagement fields live in their respective indices. A shared analytical schema common to both post indices ensures that the analytical layer can issue uniform queries across platforms:

```
{
  "content_clean":      { "type": "text"    },
  "date":               { "type": "date",   "format": "yyyy-MM-dd" },
```

```
  "created_at":          { "type": "date"    },
  "city":                { "type": "keyword" },
  "sentiment_compound": { "type": "float"   },
  "sentiment_pos":       { "type": "float"   },
  "sentiment_neg":       { "type": "float"   },
  "sentiment_neu":       { "type": "float"   },
  "sentiment_label":     { "type": "keyword" },
  "harvest_strategy":    { "type": "keyword" }
}
```

Two implementation notes are worth recording. First, `sentiment_label` is declared as a `keyword` type directly, with no automatic `.keyword` sub-field on a parent `text` type. This is the conventional choice for short categorical strings but proved to be a non-obvious integration point during API construction. Second, the `date` field is stored separately from `created_at` despite being derivable from it. We added the dedicated `date` field after discovering that BlueSky's `created_at` is missing on approximately 26.5 % of records, while `date` (populated by the ingestion pipeline from whatever timestamp source is most reliable per platform) is present on 100 % of records and is free of time-zone complications, making it the canonical join key for the city-by-day aggregations driving RQ1.

The full mapping documents for all three indices are reproduced in Appendix.

4.3 Two-Phase Data Harvesting Strategy

Data ingestion runs in two phases that share the same transformation pipeline but have different reliability and resource profiles. A one-shot **historical backfill** pre-populates each index from whatever archival window the source API exposes; a continuous **real-time streaming** layer, implemented as Fission timer-triggered functions, adds new records as they appear. Both phases write to the same Elasticsearch indices using deterministic `_id` values (Mastodon status ID, BlueSky AT-URI, or a `weather_{city}_{date}` composite), so repeated writes resolve to upserts rather than duplicates. This idempotency is relied upon throughout the system.

Historical backfill. Three Python scripts in the `database/` directory (`load_mastodon_history.py`, `load_bluesky_history.py`, `load_weather_history.py`) connect directly to Elasticsearch, page through the source API as far back as it permits, apply the shared transformation pipeline, and bulk-index the resulting documents. The scripts ran on 6 May 2026 and produced an initial corpus of approximately 9,558 Mastodon posts, 2,660 BlueSky posts, and 1,460 daily weather records spanning May 2025 through May 2026.

Real-time streaming via Fission [4]. Three Fission functions, one per source, are each triggered by an independent Fission timer (Table 2). Cadence is matched to the natural arrival rate of each source: the Mastodon public timeline produces fresh content continuously, BlueSky's rate-limited `searchPosts` arrives more sparsely, and Open-Meteo publishes only daily summaries.

On each invocation, a function authenticates to its source, fetches the next batch of new records using whatever cursor state is appropriate (Mastodon `since_id`, BlueSky `cursor`, weather `date`), applies the shared transformation pipeline described below, and bulk-indexes the transformed documents into the appropriate Elasticsearch index.

Function	Cron schedule	Cadence
`mastodon_streaming`	`*/10 * * * *`	Every 10 minutes
`bluesky_streaming`	`0 */2 * * *`	Every 2 hours
`weather_streaming`	`0 2 * * *`	Daily at 02:00 UTC

Table 2: Fission timer schedules for the three streaming functions.

Shared transformation pipeline. Each ingested record passes through the same sequence of transformations, implemented in `backend/shared/` and invoked from all three Fission functions:

1. HTML markup is stripped from the post body.
2. Language is detected and non-English posts are filtered out.
3. VADER sentiment is scored on each post, producing four numeric scores and a categorical label persisted alongside the document.
4. City attribution is attempted by case-insensitive text matching against a curated list of Australian city names and common variants. Posts with no city match retain a null `city` field.
5. A `harvest_strategy` provenance tag is attached to record how the document entered the index (which is later used by the selection-bias filter described in Section 8.1).

Performing sentiment scoring and city attribution at ingestion time (rather than at query time) means the storage layer can answer analytical queries directly through aggregations on persisted fields, with no client-side recomputation. The streaming pods have run continuously for over 21 days at the time of writing.

4.4 Data Volume and Coverage

Table 3 summarises the data holdings at the time of writing. The two post indices span 110 days of continuous social-media activity; the weather index spans approximately twelve months and extends earlier than the post window, providing a stable climatological baseline against which the post window can be situated. The "cleaned" BlueSky row applies the selection-bias filter described in Section 8.1, which excludes posts acquired through weather-keyword searches during early backfill.

Index	Documents	Date range	City-tagged
`posts_mastodon`	15,975	26 Jan – 15 May 2026	3,908
`posts_bluesky` (raw)	17,560	26 Jan – 15 May 2026	8,530
`posts_bluesky` (cleaned)	15,766	26 Jan – 15 May 2026	7,960
`weather`	1,496	1 May 2025 – 14 May 2026	1,496
Total (cleaned)	33,237	—	13,364

Table 3: Indexed data volumes and coverage.

City-tagged documents are unevenly distributed across the four target cities (Table 4), with Melbourne and Sydney dominating as Australia's two largest cities and Perth under-represented. Perth's modest sample size (364 Mastodon posts; 976 cleaned BlueSky posts) is the basis for caveats applied to the Perth-specific findings in Section 7.2.2.

City	Mastodon	BlueSky (cleaned)
Melbourne	1,416	2,579
Sydney	1,245	3,252
Brisbane	883	1,153
Perth	364	976
Total	3,908	7,960

Table 4: Distribution of city-tagged posts across the four target cities.

The relative volumes follow the expected pattern: Melbourne and Sydney dominate as Australia's two largest cities; Brisbane and Perth follow at a lower volume. The relative under-representation of Perth (approximately 9 % of Mastodon city-tagged posts and 12 % of cleaned BlueSky city-tagged posts) is the methodological basis for the modest sample size in RQ1's Perth-specific analysis discussed in Section 7.2.

5 Implementation Details

5.1 RESTful API Design

The FastAPI [7] application (`backend/api/main.py`) exposes 14 endpoints grouped by purpose (Table 5), deployed as a Kubernetes Deployment with one replica. The API follows REST conventions: nouns in URLs, query parameters for filters and pagination, and appropriate HTTP status codes (`200` on success; `400` for request-time validation errors; `422` for FastAPI parameter validation failures; `502` for upstream Elasticsearch failures). Pagination uses `limit` (capped at 500) and `offset`, delegating to Elasticsearch's `from+size` semantics.

Category	Endpoints
Health & metadata	`/api/health`, `/api/summary`
Raw retrieval	`/api/posts`, `/api/weather`, `/api/search`
Time-series	`/api/timeseries`, `/api/compare/volume`
Aggregations	`/api/sentiment`, `/api/weather/summary`, `/api/engagement`, `/api/compare/counts`, `/api/compare/sentiment`
Top-N	`/api/top-posts`, `/api/sample`

Table 5: Fourteen REST endpoints exposed by the API.

FastAPI's automatic OpenAPI 3.0 specification at `/openapi.json` makes the API self-describing. The companion HTTP client `api_client.py` (Section 5.2) was implemented against this specification alone, without consulting server source code. The API is read-only, which matches the analytical-surface semantics and removes any need to reason about idempotency or side effects on the client side.

In deployment, the REST API runs as a stateless FastAPI service inside the Kubernetes cluster. The service connects to Elasticsearch using the cluster-internal DNS name rather than `localhost`. Elasticsearch credentials are injected through Kubernetes Secrets, while index names and connection settings are supplied through environment variables in the Deployment manifest. This allows the notebook to keep a stable client-side interface through `BASE_URL = "http://localhost:8000/api"`, while the actual backend is the Kubernetes-hosted API service reached through `kubectl port-forward`.

5.2 Dual-Mode Data Access: Implementation

The architectural motivation for two interchangeable access paths is set out in Section 2.2. The two modules (`frontend/data_loader.py` and `frontend/api_client.py`) export an identical public interface: the same constants (`CITIES`, `COLORS`), the same function names (`load_posts`, `load_weather`, `daily_sentiment`, `join_posts_weather`), and the same applied filters. Internally, `data_loader` opens an Elasticsearch Point-in-Time snapshot and paginates with `search_after`, the recommended pattern for deep retrieval beyond Elasticsearch's default `max_result_window` of 10,000. `api_client` issues paginated HTTP GET requests against `/api/posts` and `/api/weather` in 500-document batches.

Filter parity is enforced inside both modules so that downstream analyses see identical data regardless of access path. In `data_loader.py` the BlueSky selection-bias filter (Section 8.1) is pushed down into Elasticsearch as a `must_not` clause; in `api_client.py` the same filter is applied client-side after fetch, because the API does not expose a filter-on-strategy parameter:

```
# api_client.py
if platform == "bluesky" and clean_bluesky:
    mask = (df["harvest_strategy"]
              .str.contains("weather", case=False, na=False))
    df = df[~mask].reset_index(drop=True)
```

The notebook selects between modules through a single import-time flag, after which all subsequent cells reference only the alias `dl`:

```
USE_API = True
if USE_API:
    import api_client as dl
else:
    import data_loader as dl
```

End-to-end verification confirms that running the notebook with `USE_API=True` and `USE_API=False` produces identical figures and numerical results. The only observable difference is that `get_overview()` reports the post-filter document count when using `data_loader.py` but returns `None` through the API path, because the API does not expose a post-filter count endpoint.

5.3 Shared Utility Modules

Three modules under `backend/shared/` are reused by both the Fission ingestion functions and the FastAPI service: `sentiment.py` (the VADER wrapper), `cities.py` (the city-attribution detector), and `transforms/` (HTML stripping, language detection, content normalisation). Co-locating this logic ensures that any change to, for example, the city-detection regex propagates uniformly without divergence between the ingestion and serving paths. All three modules are exercised by the test suite described in Section 6.

6 Automated Testing and Continuous Deployment

Software testing and deployment for this project follows two principles. First, **tests run without a live cluster**: every test executes in a vanilla Python container in seconds, with no dependence on a running

Elasticsearch instance or Kubernetes cluster. Second, **tests are exercised on every change**: a GitLab CI/CD pipeline runs the full test suite and automates deployment on each push and merge request, so that broken code cannot land on the `main` branch unnoticed. The infrastructure consists of seven test modules, a shared fixture file, and a single `.gitlab-ci.yml` pipeline definition.

6.1 Test Suite Organisation

Tests live in the top-level `test/` directory; a shared `conftest.py` ensures imports resolve correctly. The suite separates into **unit tests** for pure functions (five modules) and **contract tests** for the FastAPI surface using the `FakeES` mock (Table 6). Each module maps one-to-one to an application module so a failing test points immediately to the broken source file; `test_api.py` is the largest, reflecting the API's central position in the architecture.

Module	Layer tested	Type	LOC
`test_transforms.py`	HTML strip, language, normalise	Unit	136
`test_cities.py`	City attribution	Unit	22
`test_keywords.py`	Weather-keyword detection	Unit	28
`test_sentiment.py`	VADER wrapper	Unit	20
`test_weather_codes.py`	Open-Meteo code translation	Unit	14
`test_api.py`	FastAPI endpoints + ES contract	Contract	256
`conftest.py`	Shared fixtures, import setup	Contract	9
Total			**485**

Table 6: Test modules by type, with approximate line counts.

6.2 The FakeES Mock Pattern

Testing the FastAPI application without a live Elasticsearch cluster is handled through a minimal `FakeES` class (approximately 60 lines) that imitates only the client methods the application actually uses: `ping`, `count`, `search`, and `indices.exists`. Tests replace the live client with a `FakeES` instance loaded with fixture data, then exercise the endpoint through FastAPI's `TestClient`:

```
def test_sentiment_endpoint_returns_distribution():
    fixtures = {"posts_mastodon": [
        {"sentiment_label": "positive"},
        {"sentiment_label": "positive"},
        {"sentiment_label": "negative"},
    ]}
    app.dependency_overrides[get_es] = lambda: FakeES(fixtures)
    client = TestClient(app)
    r = client.get("/api/sentiment?platform=mastodon")
    assert r.status_code == 200
    assert r.json()["sentiment_distribution"] == {
        "positive": 2, "negative": 1
    }
```

The full suite runs in approximately one second locally and twenty seconds in CI, tests are deterministic with no shared global state, and `FakeES` is small enough that extending it for a new endpoint is a

matter of adding one stub method per new aggregation pattern. The unavoidable trade-off is that schema mismatches between `FakeES` and the real Elasticsearch are not caught; the `.keyword` field-name bug discussed in Section 8.1 is exactly the kind of bug this gap permits.

6.3 GitLab CI/CD Pipeline

The pipeline runs three sequential stages on every push and merge request: **Test**, **Build**, and **Deploy**. The Test stage installs dependencies in a slim Python container and runs `pytest -q` over all seven test modules; pipeline execution halts on any failure. The Build stage compiles the FastAPI service into a container image and pushes it to GitLab Container Registry, tagged with both the commit SHA and `latest`. The Deploy stage, restricted to the `main` branch, applies the updated Kubernetes manifests against the cluster using a kubeconfig stored as a CI variable. Total pipeline runtime is approximately 90 seconds for Test, plus additional time for Build and Deploy.

Every commit on `main` at the time of writing has a passing pipeline. The Fission ingestion functions are currently deployed through a separate `./deploy.sh` script rather than through the CI pipeline.

6.4 What the Pipeline Verifies (and What It Does Not)

The pipeline verifies three classes of properties: shape-of-data invariants (HTML stripping, language detection, normalisation), pure-function correctness (sentiment scoring, city attribution, keyword detection, weather code translation), and API contract (endpoint response shapes and status codes via FakeES).

Three categories of behaviour are intentionally outside the automated suite. Live Elasticsearch integration is left to manual testing during development. Kubernetes deployment health is verified manually through the operational commands not through an end-to-end probe from CI. Cron-driven Fission streaming behaviour is exercised in production by the timer triggers themselves; the function-handler code is overwhelmingly composed of calls into the shared transformation modules, and unit-testing the boilerplate would yield little incremental confidence.

7 Frontend: Analytical Scenarios and Results

The frontend analytical layer is implemented as a Jupyter notebook that consumes indexed data through the dual-mode access architecture described in Section 5.2. The notebook serves as the primary analytical client of the deployed system and generates the visualisations and statistical analyses presented in this section.

The analyses below are designed not only to explore the collected social-media datasets, but also to exercise the deployed cloud-native pipeline end-to-end, including Elasticsearch aggregation, REST API access, city-level attribution, and cross-platform data integration.

7.1 Scenario Selection Rationale

Two complementary analytical scenarios were selected to evaluate both the engineering capabilities of the deployed system and the analytical value of the collected datasets:

- **Scenario 1.** Do weather conditions correlate with daily aggregate social-media sentiment in Australian cities?
- **Scenario 2.** Do Mastodon and BlueSky users differ in sentiment expression and discourse vocabulary within the Australian context?

The selected scenarios are complementary by design. Scenario 1 exercises the complete weather-integration workflow, including weather indexing, city-level aggregation, and sentiment analysis, while evaluating whether aggregate online sentiment varies with environmental conditions.

Scenario 2 focuses on cross-platform comparison and examines differences in sentiment expression and discourse vocabulary between Mastodon and BlueSky users within the Australian context.

Together, the two scenarios exercise all fourteen REST endpoints, all three Elasticsearch indices, and the city-attribution pipeline.

7.2 Scenario 1: Weather–Sentiment Correlation

7.2.1 Methodology

The analysis joins city-tagged posts with daily weather observations on the date dimension. The methodology proceeds in four steps:

1. **Filter to city-tagged posts.** Restrict to posts whose `city` field matches one of the four target cities. BlueSky posts also have the selection-bias filter applied (Section 8.1). The resulting subset is 3,908 Mastodon and 7,960 BlueSky posts.
2. **Aggregate to daily city-level sentiment.** For each (city, date) cell with at least three posts, compute the mean VADER compound score.
3. **Join with weather.** Inner-join each daily sentiment cell with the weather observation for the same (city, date).
4. **Compute correlations.** For each city, compute Spearman rank correlations between (i) temperature and sentiment, and (ii) precipitation and sentiment.

7.2.2 Findings

The weather observations exhibit clear seasonal and regional variation across the four target cities over the January–May 2026 observation window. Temperatures generally decline towards late autumn across all cities, while precipitation remains highly uneven across both time and location.

Figures 2 and 3 visualise the daily temperature and precipitation patterns respectively.

To evaluate whether these weather variations correspond to changes in online sentiment, daily aggregate sentiment was compared against temperature and precipitation separately for each city.

Figures 4 and 5 present the resulting weather–sentiment relationships.

The resulting Spearman rank correlations are summarised in Figure 6 and Table 7.

Three findings emerge from the analysis.

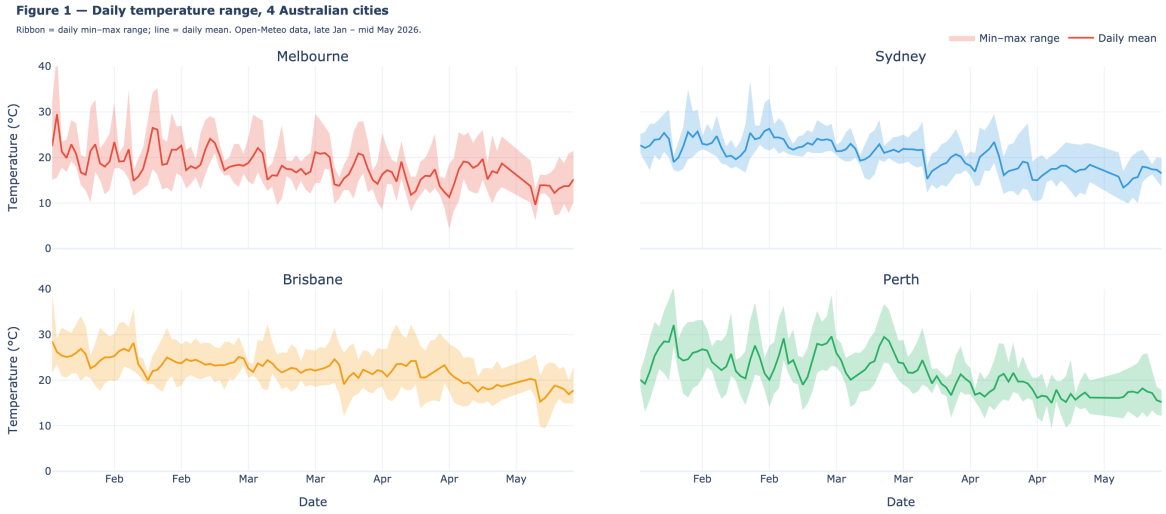


Figure 2: Daily mean temperature across the four target cities (Melbourne, Sydney, Brisbane, Perth) over the observation window.

City	Temp ρ	p	Precip ρ	p
Melbourne	$+0.08$	0.51	-0.04	0.74
Sydney	$+0.05$	0.66	$+0.11$	0.32
Brisbane	-0.11	0.42	-0.07	0.61
Perth	$\mathbf{-0.42}$	**0.02**	$+0.16$	0.39

Table 7: Spearman correlations for temperature and precipitation against daily aggregate sentiment, by city. Significant cells ($p < 0.05$) are bolded.

First, weather–sentiment correlations remain weak across three of the four cities, with all correlation magnitudes below $|\rho| < 0.12|$. This suggests that daily weather variation has limited explanatory power over aggregate online sentiment at the city level.

Second, Perth exhibits the only statistically significant relationship: a moderate negative association between temperature and sentiment ($\rho = -0.42$, $p = 0.02$). One possible explanation is that Perth's distinct climatic patterns produce stronger behavioural or posting responses during periods of extreme heat.

Third, precipitation demonstrates no statistically significant relationship with sentiment in any city. The absence of a consistent rainfall effect suggests that broader discourse topics and current events likely dominate short-term sentiment dynamics more strongly than local weather conditions.

Overall, the findings indicate that weather exerts only limited influence on aggregate social-media sentiment at daily resolution. The Perth result should nevertheless be interpreted with caution due to its smaller effective sample size (364 Mastodon and 976 BlueSky posts).

7.3 Scenario 2: Cross-Platform Discourse Comparison

7.3.1 Methodology

The cross-platform analysis asks whether Mastodon and BlueSky users within the Australian context differ in (i) the sentiment characteristics of their posts and (ii) the vocabulary of their discourse. The

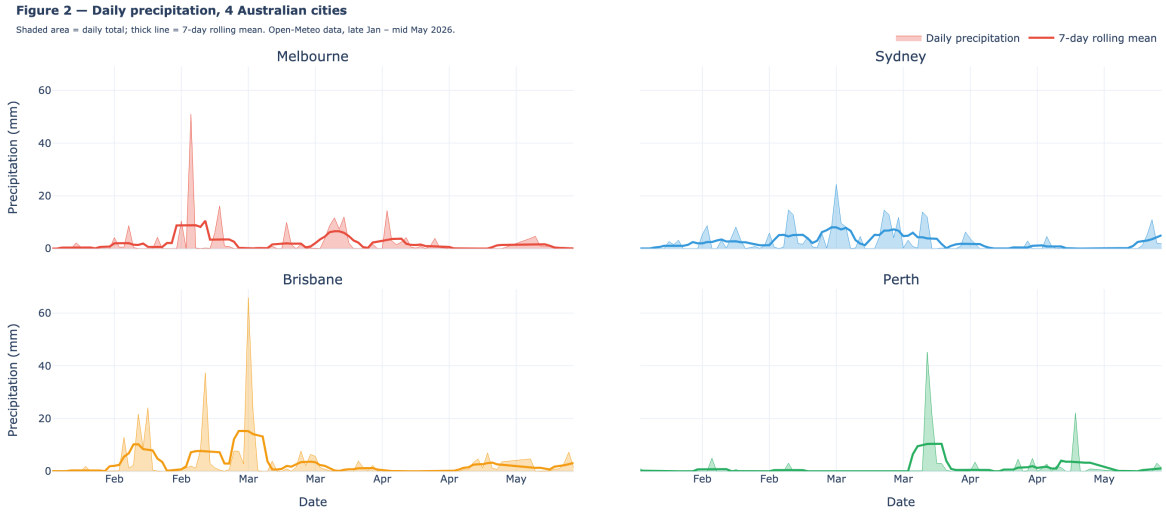


Figure 3: Daily precipitation across the four target cities.

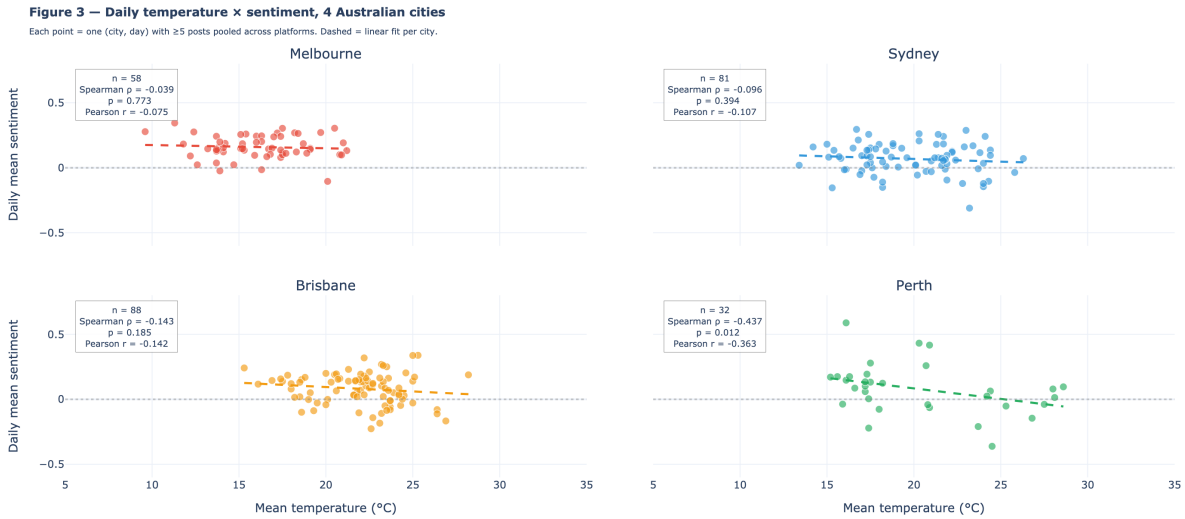


Figure 4: Daily aggregate sentiment versus daily mean temperature, by city.

analysis proceeds in four components:

1. **Vocabulary extraction.** For each platform, extract the top-100 most frequent content tokens after stop-word removal. The stop-word list combines standard English stop-words with a custom list of social-media artefacts (`https`, `co`, `amp`, `rt`, `via`) and platform-specific URL fragments. The list was refined through two iterations after early word clouds were dominated by these artefacts.

2. **Sentiment distribution comparison.** Aggregate sentiment label distributions across each platform using the `/api/sentiment` endpoint, comparing the proportion of positive, neutral, and negative posts as well as the mean and variance of the compound score.

3. **Daily sentiment time series.** For each platform, compute the daily mean compound sentiment over the observation window using the `/api/timeseries` endpoint, and inspect whether the two platforms move together or exhibit decoupled patterns.



Figure 5: Daily aggregate sentiment versus daily precipitation, by city.

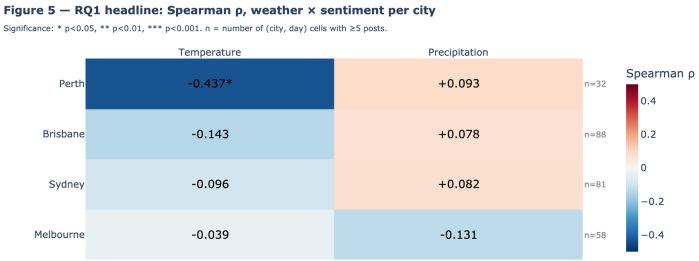


Figure 6: Spearman rank correlation heat-map between weather variables and daily aggregate sentiment, by city.

4. **Platform-specific weather–sentiment regressions.** Repeat the daily Spearman rank correlation between aggregate sentiment and weather variables (temperature, precipitation) independently for each platform's city-tagged subset. This tests whether the null result observed in Scenario 1 is robust across platforms or specific to one of them.

Together, these four components let us distinguish *vocabulary* differences (what the platforms discuss) from *sentiment* differences (how positively or negatively they discuss it).

7.3.2 Findings

The cross-platform comparison reveals substantial differences in discourse vocabulary between Mastodon and BlueSky users, despite broadly similar aggregate sentiment levels.

Figure 7 visualises the most frequent tokens observed on each platform.

The vocabulary divergence is especially visible in politically charged terms such as "Trump", "Labor", "Albanese", and "election", which appear prominently on BlueSky but are much less common on Mastodon. Conversely, Mastodon contains more location names, repost artefacts (e.g. "via"), and automated news-style language. These patterns suggest that the two platforms sample different discourse communities within the Australian online environment.

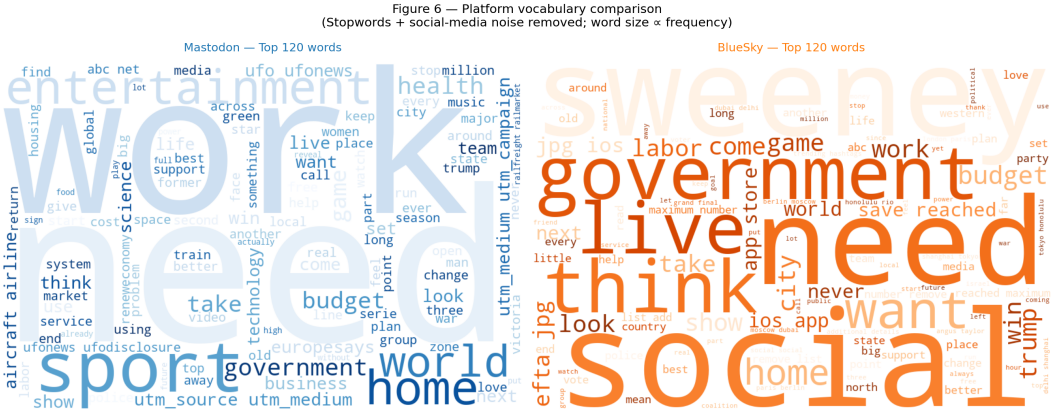


Figure 7: Word clouds of the top-100 most frequent tokens on each platform (left: Mastodon; right: BlueSky).

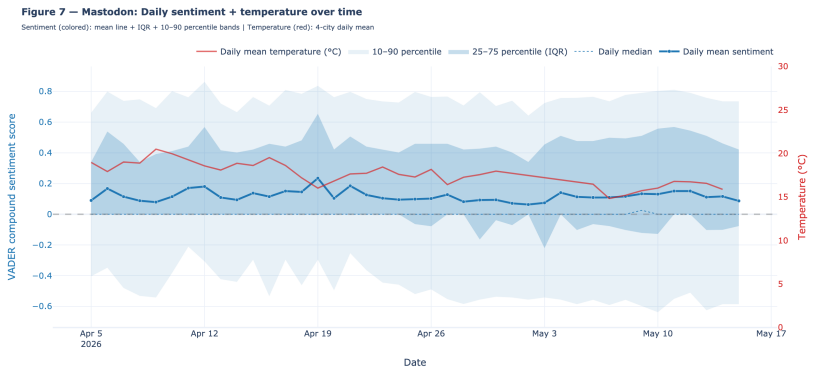


Figure 8: Mastodon daily mean compound sentiment over the observation window.

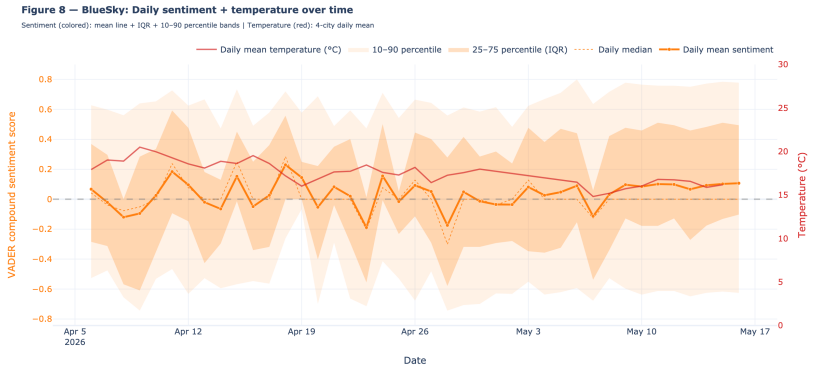


Figure 9: BlueSky daily mean compound sentiment over the observation window.

Figures 8 and 9 illustrate the temporal evolution of daily mean sentiment across the two platforms.

Both platforms exhibit mildly positive daily mean sentiment ($\bar{c} \approx 0.10$–0.15) with broadly similar variance over time. However, their categorical sentiment distributions differ noticeably. Mastodon contains a larger neutral share ($\sim$35%), likely reflecting the prevalence of automated news aggregation accounts, while BlueSky exhibits a more balanced three-way split between positive, neutral, and negative posts.

Figures 10 through 13 present platform-specific weather–sentiment relationships.

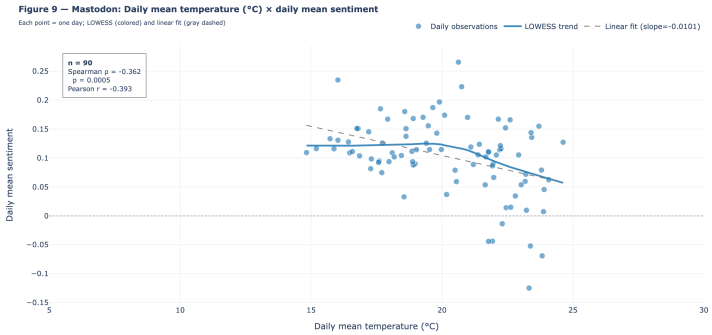


Figure 10: Mastodon daily sentiment versus daily mean temperature.

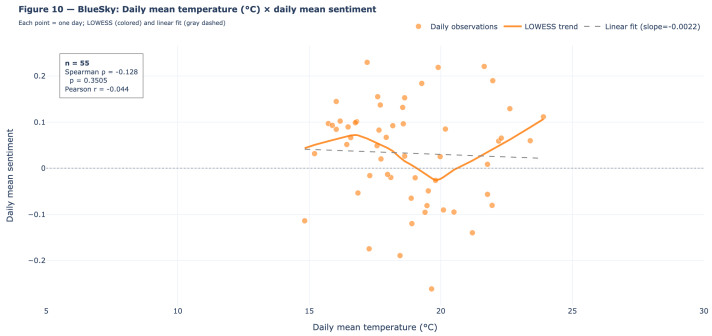


Figure 11: BlueSky daily sentiment versus daily mean temperature.

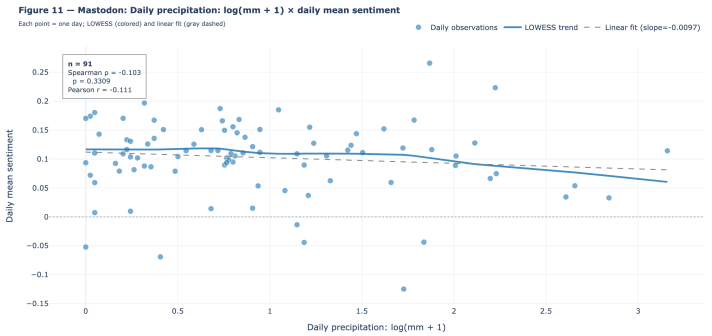


Figure 12: Mastodon daily sentiment versus daily precipitation.

The platform-specific plots further confirm that the weak weather–sentiment relationships identified in Scenario 1 are not artefacts of city-level aggregation.



Figure 13: BlueSky daily sentiment versus daily precipitation.

Overall, the principal finding is that the two platforms differ more strongly in discourse vocabulary than in aggregate sentiment magnitude. While both platforms maintain broadly similar average sentiment levels, they capture distinct styles of online discussion and different social-media subcommunities.

7.4 Synthesis Across Scenarios

Taken together, the two analytical scenarios reveal that weather conditions exert only limited influence on aggregate city-level social-media sentiment within the observation window.

Scenario 1 demonstrates that correlations between weather variables and sentiment remain weak across most cities, with the only statistically significant relationship observed in Perth, where higher temperatures are associated with lower sentiment scores.

Scenario 2 further shows that cross-platform differences are more strongly reflected in discourse vocabulary than in sentiment magnitude. While Mastodon and BlueSky exhibit broadly similar average sentiment levels, the platforms capture different styles of online discussion and distinct discourse communities.

Collectively, these findings suggest that broader social and political discourse plays a more substantial role in shaping online sentiment than short-term local weather variation. Methodological limitations and interpretive caveats are discussed in Section 8.4.

8 Discussion

8.1 Error Handling and Engineering Challenges

The error-handling philosophy adopted throughout this project is to address the failure mode through a structural fix when one exists, document the limitation honestly when one does not, and never silence symptoms. Five distinct engineering challenges arose during development.

BlueSky session rate-limiting. The first deployment of the `bluesky_streaming` function authenticated to BlueSky on every timer invocation by calling `com.atproto.server.createSession`, which is rate-limited per account. After a few invocations the function began receiving `429 Too Many Requests` responses, and the BlueSky index plateaued at approximately 2,849 posts. The fix is a persistent JWT cache backed by a Kubernetes ConfigMap (`bluesky-jwt`) storing the current access JWT, refresh JWT, and access-JWT expiry. The function calls `createSession` only once per account lifetime, thereafter

calling the unthrottled `refreshSession` endpoint as access tokens approach expiry and patching the ConfigMap in place. A narrowly-scoped RBAC Role grants the function's ServiceAccount only `get` and `patch` permission on that single ConfigMap, preventing any privilege creep. After deployment on 9 May 2026 the BlueSky index resumed steady growth, reaching 17,560 posts at the time of writing, approximately a six-fold increase. The pattern is reusable: any serverless function requiring authenticated session state across invocations can apply the same ConfigMap-plus-RBAC mechanism without introducing an additional database service.

Selection bias in search-acquired BlueSky posts. An early BlueSky backfill ran `app.bsky.feed.searchPosts` queries with weather-related keywords (`heatwave`, `auswx`, `polar blast`) to bootstrap weather-relevant content, tagging the resulting documents with `harvest_strategy` values such as `search:heatwave`. This strategy worked for its original purpose but introduced a non-trivial selection bias: roughly 10 % of the BlueSky corpus was disproportionately weather-oriented relative to organic discourse, which would have distorted the cross-platform sentiment comparison. The fix is a query-time filter applied uniformly in both data-access modules (`data_loader.py` pushes it into Elasticsearch as a `must_not` clause; `api_client.py` applies it client-side after fetch) that excludes posts whose `harvest_strategy` matches any weather-search provenance value. The unfiltered index is preserved for analyses that might specifically benefit from it; the cleaned subset of 15,766 posts is what every figure in Section 7 draws upon.

City attribution without reliable geo metadata. Neither Mastodon nor BlueSky provides reliable per-post geographic metadata: Mastodon exposes no geo field, and BlueSky's user-profile location string is free-text, inconsistently populated, and attached to the user rather than the post. Because the city-level aggregation underpinning RQ1 requires some attribution mechanism, we implemented text-based detection in `backend/shared/cities.py`: each post is tested case-insensitively against a curated list of Australian city names and common variants ("Melbourne", "Naarm", "Mel.", and so on), with word-boundary anchors to avoid spurious substring matches. The detector is intentionally conservative, producing no tag for the majority of posts but high precision on those tagged: a post explicitly mentioning "Melbourne" is overwhelmingly likely to be about Melbourne. The resulting attribution rates are approximately 26 % on Mastodon and 49 % on cleaned BlueSky, restricting RQ1 to 11,868 attributed posts across the four cities. This is a deliberate precision-over-recall trade-off whose sample-size consequence (notably Perth's 364 Mastodon posts) is acknowledged in Section 8.4.

The `.keyword` field-name bug. During integration of `api_client.py`, the aggregation endpoints `/api/sentiment` and `/api/weather/summary` silently returned empty distribution dictionaries. The underlying queries referenced `sentiment_label.keyword`, the standard syntax when Elasticsearch text fields carry an automatic `keyword` sub-field. Our mappings, however, declare these fields as `keyword` directly, so `.keyword` resolved to a non-existent path and the aggregations returned empty buckets without error. Direct verification against the live mapping with `curl /_mapping/field/sentiment_label` confirmed the hypothesis; the fix removed the `.keyword` suffix from three aggregation references in `main.py`. The broader lesson is that direct schema verification is invariably faster than reasoning by convention about how a schema "should" look.

The stale-pod deployment gap. After the `.keyword` fix was merged into `main`, the API continued returning empty distributions for 28 hours. The corrected image was correctly published to the GitLab Container Registry under the `:latest` tag, but the running pod retained the image digest fixed at its creation time. Kubernetes does not automatically detect or re-pull updated images for existing pods; the registry-to-pod gap must be bridged by an explicit rollout. The fix was the single command `kubectl rollout restart deployment/ccc-rest-api`, after which the new pod pulled the current image (since `imagePullPolicy` is `Always`) and the API immediately began returning populated buckets. The lesson, recorded here because it bears repetition, is that merging a fix into `main` is not synonymous with deploying it. A GitOps controller such as ArgoCD or Flux would close this gap by watching the registry and triggering rollouts automatically, an extension noted in Section 8.

8.2 Fault Tolerance and Single Points of Failure

The deployed system's fault tolerance falls out of its architectural choices rather than from explicit redundancy. Deterministic document `_id` values make ingestion idempotent, so retries, timer overlaps, and partial-batch restarts resolve to upserts rather than duplicates. The Fission functions and the FastAPI service are stateless and freely replaceable; the state that must persist (BlueSky session tokens, indexed documents, cluster configuration) is externalised to the JWT ConfigMap, the Cinder-backed Elasticsearch PVC, and the control plane's `etcd` respectively. Several single points of failure remain and are acknowledged honestly: a single-node Elasticsearch cluster, a single control-plane VM, a single-replica FastAPI Deployment, and a single BlueSky account in the JWT cache. Loss of any of these produces a brief unavailability window rather than data loss, and multi-replica or multi-account configurations were judged unnecessary for the assessment window. The operational evidence is the cluster itself: at the time of writing the Mastodon and weather Fission pods have run for over 21 days with zero unexpected restarts, and the BlueSky pod has run continuously since the JWT cache was deployed on 9 May 2026.

8.3 Pros and Cons of the Cloud Technology Stack

The four principal technologies in the stack (the Melbourne Research Cloud, Kubernetes, Fission, and Elasticsearch with the ECK operator) each contributed distinctive strengths to the project. MRC's Magnum service produced a working Kubernetes cluster within a few minutes through a web interface, with Cinder block storage integrating seamlessly into the PVC abstraction. Kubernetes itself delivered the highest engineering leverage of the four: every cluster resource lives as a version-controlled YAML manifest, making the cluster's full configuration reproducible from the repository alone and auditable through Git history. Fission's pool-manager strategy keeps cold-start latency effectively zero from the developer's perspective, and its CLI (`fission fn create`) shortens the deployment feedback loop substantially relative to building and pushing container images by hand. ECK reduces what could have been a multi-day Elasticsearch operational setup (certificate authority, cluster bootstrap, version management) to a single YAML resource, and Elasticsearch's DSL is expressive enough that the analytical endpoints of our API delegate aggregations to the storage layer rather than computing them on the client.

The friction points encountered live mostly at boundaries between technologies rather than within any single one of them. MRC does not expose Kubernetes LoadBalancer or Ingress routes, so every external access required an authenticated `kubectl port-forward` session, which silently dies when the

underlying pod restarts. Kubernetes does not bridge the gap between "the registry image has changed" and "the running pod should use it", which produced the stale-pod incident (Section 8.1); a GitOps controller would close this gap automatically. Fission's builder stage can take several minutes on the first deployment of a function with new Python dependencies, and its function logs are accessible only for the most recent invocation, with no persistent log aggregation configured. Elasticsearch's default `max_result_window` of 10,000 caught us unprepared during full-corpus retrieval, and the `keyword`-versus-`text` mapping convention proved to be a subtle integration trap (Section 8.1). None of these frictions was prohibitive, and we consider the stack appropriately matched to the assignment's scope and schedule.

8.4 Limitations and Future Work

Three limitations of the deployed system are worth flagging briefly. VADER is a lexicon-based sentiment scorer with known weaknesses on sarcasm, irony, dry humour, and Australian English slang, all of which are common in the corpus; the null-result interpretation of RQ1 should therefore be read as "no signal above VADER's noise floor at daily aggregate resolution" rather than "no relationship exists in the underlying phenomenon". The conservative city-attribution detector (Section 8.1) tags only 26 % of Mastodon and 49 % of cleaned BlueSky posts, which trades recall for precision and constrains per-city sample sizes (notably Perth, with 364 Mastodon posts). And the 110-day observation window suffices for within-window cross-platform comparison but does not cover a complete annual seasonal cycle, putting quarterly and year-over-year analyses out of scope.

Two engineering extensions stand out as the most impactful follow-up work. A GitOps controller such as ArgoCD or Flux would watch the GitLab Container Registry and trigger Kubernetes rollouts automatically on each new image digest, closing the merge-to-deploy gap that produced the stale-pod incident (Section 8.1). A cluster-wide monitoring stack (Prometheus for metric collection plus Grafana for visualisation, deployable as a single Helm chart) would replace the manual `kubectl get pods` workflow currently used to verify operational health, and combined with AlertManager rules for ingestion stalls would close the largest operational vulnerability remaining at submission.

9 Conclusions

This project delivered a cloud-native data analytics application that ingests, indexes, and analyses social-media posts from Mastodon and BlueSky alongside daily weather observations for four Australian cities. The system runs on a four-node Kubernetes cluster on the Melbourne Research Cloud, comprising three Fission ingestion functions, an ECK-managed three-index Elasticsearch store, a FastAPI service exposing fourteen REST endpoints, and a Jupyter notebook that exercises the full data path through a dual-mode access layer. At submission the indices hold over 33,000 documents, the streaming pods have run continuously without restart for over three weeks, and an automated test suite of approximately 500 lines is executed on every push through a GitLab CI/CD pipeline that also builds and deploys the API container on every merge to `main`.

Three architectural patterns from the project are worth carrying forward: the dual-mode data-access layer, which makes the RESTful interface a live and demonstrable path through a single-flag switch in the analytical client; the persistent JWT cache, which uses native Kubernetes ConfigMap and RBAC

primitives to hold cross-invocation session state for a rate-limited external API without introducing an additional database service; and the FakeES mock pattern, which keeps CI fast and independent of cluster availability at the cost of a documented gap that manual integration testing fills. The team's broader takeaway from the technology stack is that the declarative-and-immutable discipline embodied by Kubernetes, ECK, and version-controlled manifests is the single most valuable engineering property of the toolchain, and that the friction points encountered live at boundaries between technologies rather than within any single one of them.

10 Team Contribution

Table 8: Team member contributions

Member	Primary Responsibilities
Xianglong Fu	Implemented all three Fission streaming pipelines including the shared transformation pipeline (`backend/shared/`, sentiment scoring, language detection, city attribution). K8s declarative resources, BlueSky rate-limit mitigation, Scenario 2 analysis (Mastodon vs BlueSky), report writing.
Muchuan Ji	Cluster environment setup and maintenance on MRC; kubeconfig and `kubectl` access support for team members; FastAPI REST backend implementation and debugging; Docker image build and GitLab Container Registry deployment; Kubernetes Deployment and Service configuration for the REST API; integration testing through port-forwarding and Jupyter Notebook access.
Zhuo Wen	Scenario 1 analytical prototype; weather-sentiment methodology design (Spearman's rank correlation, city-by-day aggregation strategy, sample-size thresholds); early data exploration and feature-engineering decisions on the weather index; co-authoring of the Scenario 1 narrative, report writing.
Zeming Ji	Automated test suite design and implementation; the FakeES mock pattern for FastAPI contract testing; GitLab CI/CD pipeline configuration; test coverage for shared transformation modules; documentation of the test architecture.
Chang Gu	supporting analytical work including the Elasticsearch Index Analytics Report notebook (`frontend/Overview.ipynb`); Scenario 1 methodology review and validation; visual design of analytical figures including the correlation heatmap and word clouds; report proofreading and consistency checking.

11 LLM Usage Declaration

The team used Anthropic Claude for code assistance, repository structure design, and report drafting. OpenAI ChatGPT was used in the early stages of the project to brainstorm and refine the research direction. All architectural decisions, deployed code, and numerical results in this report were reviewed and verified by the team before inclusion.

References

[1] Kubernetes Documentation. `https://kubernetes.io/docs/`, 2026. Accessed: 2026-05-18.

[2] Australian Research Data Commons. Nectar Research Cloud. `https://ardc.edu.au/services/nectar-research-cloud/`, 2026. Accessed: 2026-05-18.

[3] Bluesky PBC. AT Protocol Documentation. https://atproto.com/, 2026. Accessed: 2026-05-18.

[4] Fission Authors. Fission: Serverless Functions for Kubernetes. https://fission.io/docs/, 2026. Accessed: 2026-05-18.

[5] Mastodon gGmbH. Mastodon API Documentation. https://docs.joinmastodon.org/, 2026. Accessed: 2026-05-18.

[6] Open-Meteo. Open-Meteo Free Weather API. https://open-meteo.com/, 2026. Accessed: 2026-05-18.

[7] Sebastián Ramírez. FastAPI. https://fastapi.tiangolo.com/, 2026. Accessed: 2026-05-18.

A Source Code Repository

GitLab URL	`https://gitlab.unimelb.edu.au/muchuanj/comp90024_team_47`
Visibility	Private (teaching team access granted)
Cluster name	`comp90024` (on Melbourne Research Cloud)
Kubeconfig	`config.47.yaml` (shared separately)

B YouTube Video Link for Demonstration

YouTube URL	`https://youtu.be/stVT7wIy0Zs`
Duration	approximately 10 minutes
Access	unlisted (link-only, no public listing)

C Elasticsearch Mappings

This appendix reproduces the full Elasticsearch mappings for the three indices that comprise the storage layer, as committed to the repository under `database/mappings/`.

C.1 posts_mastodon

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "index.max_result_window": 50000
  },
  "mappings": {
    "properties": {
      "post_id":            { "type": "keyword" },
      "url":                { "type": "keyword" },
      "content_raw":        { "type": "text" },
      "content_clean":      { "type": "text" },
      "language":           { "type": "keyword" },
      "created_at":         { "type": "date" },
      "date":               { "type": "date", "format": "yyyy-MM-dd" },
      "author_acct":        { "type": "keyword" },
      "author_display_name":{ "type": "text" },
      "author_followers":   { "type": "integer" },
      "favourites_count":   { "type": "integer" },
      "reblogs_count":      { "type": "integer" },
      "replies_count":      { "type": "integer" },
      "instance":           { "type": "keyword" },
      "city":               { "type": "keyword" },
      "sentiment_compound": { "type": "float" },
      "sentiment_pos":      { "type": "float" },
      "sentiment_neg":      { "type": "float" },
      "sentiment_neu":      { "type": "float" },
      "sentiment_label":    { "type": "keyword" },
      "harvest_strategy":   { "type": "keyword" },
```

```
      "ingested_at":        { "type": "date" }
    }
  }
}
```

C.2 posts_bluesky

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "index.max_result_window": 50000
  },
  "mappings": {
    "properties": {
      "post_uri":           { "type": "keyword" },
      "post_cid":           { "type": "keyword" },
      "content_raw":        { "type": "text" },
      "content_clean":      { "type": "text" },
      "language":           { "type": "keyword" },
      "created_at":         { "type": "date" },
      "date":               { "type": "date", "format": "yyyy-MM-dd" },
      "author_did":         { "type": "keyword" },
      "author_handle":      { "type": "keyword" },
      "author_display_name":{ "type": "text" },
      "like_count":         { "type": "integer" },
      "repost_count":       { "type": "integer" },
      "reply_count":        { "type": "integer" },
      "city":               { "type": "keyword" },
      "sentiment_compound": { "type": "float" },
      "sentiment_pos":      { "type": "float" },
      "sentiment_neg":      { "type": "float" },
      "sentiment_neu":      { "type": "float" },
      "sentiment_label":    { "type": "keyword" },
      "harvest_strategy":   { "type": "keyword" },
      "ingested_at":        { "type": "date" }
    }
  }
}
```

C.3 weather

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "index.max_result_window": 50000
  },
  "mappings": {
    "properties": {
      "city":                  { "type": "keyword" },
```

```
      "date":                 { "type": "date", "format": "yyyy-MM-dd" },
      "latitude":             { "type": "float" },
      "longitude":            { "type": "float" },
      "temp_min":             { "type": "float" },
      "temp_mean":            { "type": "float" },
      "temp_max":             { "type": "float" },
      "apparent_temp_max":    { "type": "float" },
      "apparent_temp_min":    { "type": "float" },
      "precipitation_mm":     { "type": "float" },
      "rain_mm":              { "type": "float" },
      "humidity":             { "type": "float" },
      "wind_speed_max":       { "type": "float" },
      "weather_code":         { "type": "integer" },
      "weather_condition":    { "type": "keyword" },
      "is_rainy":             { "type": "boolean" },
      "is_heavy_rain":        { "type": "boolean" },
      "is_extreme_hot":       { "type": "boolean" },
      "is_extreme_cold":      { "type": "boolean" },
      "ingested_at":          { "type": "date" }
    }
  }
}
```

D Sample Elasticsearch Queries

D.1 Selection-Bias Filter for Cross-Platform Comparison

```
{
  "query": {
    "bool": {
      "must": [
        { "match_all": {} }
      ],
      "must_not": [
        { "wildcard": { "harvest_strategy": "*weather*" } },
        { "term":     { "harvest_strategy": "search:heatwave" } },
        { "term":     { "harvest_strategy": "search:polar blast" } },
        { "term":     { "harvest_strategy": "search:auswx" } }
      ]
    }
  }
}
```

D.2 Sentiment Distribution Aggregation

```
{
  "size": 0,
  "query": { "match_all": {} },
  "aggs": {
    "sentiment_distribution": {
```

```
      "terms": {
        "field": "sentiment_label",
        "size":  10
      }
    },
    "average_sentiment": {
      "avg": { "field": "sentiment_compound" }
    },
    "min_sentiment": {
      "min": { "field": "sentiment_compound" }
    },
    "max_sentiment": {
      "max": { "field": "sentiment_compound" }
    }
  }
}
```

D.3 City-Date Sentiment Aggregation (RQ1 Preparation)

```
{
  "size": 0,
  "query": {
    "bool": {
      "must": [
        { "terms": { "city": ["Melbourne","Sydney","Brisbane","Perth"] } }
      ]
    }
  },
  "aggs": {
    "by_city": {
      "terms": { "field": "city", "size": 4 },
      "aggs": {
        "by_date": {
          "date_histogram": {
            "field":             "date",
            "calendar_interval": "1d",
            "min_doc_count":     3
          },
          "aggs": {
            "mean_compound": {
              "avg": { "field": "sentiment_compound" }
            }
          }
        }
      }
    }
  }
}
```

D.4 Deep Pagination via Point-in-Time

```
POST /posts_mastodon/_pit?keep_alive=1m
```

Then issue successive `_search` requests, carrying the PIT ID and the last document's sort values as `search_after`:

```
{
  "size": 500,
  "query": { "match_all": {} },
  "sort": [
    { "ingested_at": "asc" },
    { "_shard_doc":  "asc" }
  ],
  "pit": { "id": "<pit-id-from-previous-call>", "keep_alive": "1m" },
  "search_after": [
    "<last-ingested_at-value>",
    "<last-shard-doc-value>"
  ]
}
```

E Repository Structure

Listing 1: Project file layout, showing key source files

```
COMP90024_team_47/
|-- README.md
|-- .gitlab-ci.yml                          # CI/CD pipeline (Test/Build/Deploy)
|-- .gitignore
|-- .env.example
|
|-- frontend/                               # Jupyter analytical client
|   |-- comp90024_team_47_analysis.ipynb    # Primary integrated notebook
|   |-- data_loader.py                      # ES direct access (PIT pagination)
|   |-- api_client.py                       # HTTP access via REST API
|   |-- Overview.ipynb                      # ES index inventory report
|   `-- (legacy prototype notebooks)
|
|-- backend/
|   |-- api/
|   |   |-- main.py                         # FastAPI service, 14 endpoints
|   |   `-- Dockerfile
|   |-- fission/                            # 3 Fission timer-triggered functions
|   |   |-- mastodon_streaming/main.py
|   |   |-- bluesky_streaming/main.py
|   |   `-- weather_streaming/main.py
|   `-- shared/                             # Reusable modules (used by all above)
|       |-- sentiment.py                    # VADER wrapper
|       |-- cities.py                       # City attribution
|       |-- keywords.py                     # Weather-keyword detection
|       |-- weather_codes.py
|       |-- es_client.py
|       `-- transforms/
|           |-- mastodon.py
```

```
|           `-- bluesky.py
|
|-- database/
|   |-- mappings/                        # ES index schemas
|   |   |-- posts_mastodon.json
|   |   |-- posts_bluesky.json
|   |   `-- weather.json
|   |-- queries/                         # Documented query patterns
|   |-- load_mastodon_history.py         # One-shot historical backfill
|   |-- load_bluesky_history.py
|   `-- load_weather_history.py
|
|-- k8s/                                 # Kubernetes manifests
|   |-- api-deployment.yaml              # FastAPI Deployment + Service
|   |-- configmaps.yaml                  # bluesky-jwt ConfigMap
|   |-- secrets.example.yaml             # Secret template
|   `-- rbac/
|       `-- bluesky-jwt-rbac.yaml        # ServiceAccount + Role + RoleBinding
|
|-- test/                                # Automated test suite (478 LOC)
|-- data/                                # Cached fixtures and aggregations
|-- docs/                                # PDF report, figures, architecture diagram
`-- llm/                                 # LLM usage declaration
```